

Least Squares

Overview

03 Feb 2026

- Extend the 1-D DFT to higher dimensions, and understand two-dimensional frequencies
- Get a geometric intuition of projection onto a subspace.
- Learn about least squares optimization and how they are solved using the Normal Equation.
- Learn about weighted least squares and how the normal equation is modified.
- How to use numerical libraries to solve linear least squares problems.
- How to generalize least squares to higher order polynomials as well as exponential and power fits.

Async

2D FFT

2D Discrete
Fourier
Transform

- a **vector space** is a space where you can **add** and **scale elements**
- **vectors** are just **elements** of a **vector space**
- two **identify orthogonal** elements, **inner product** is needed
- **2D arrays (tensors)** are **vectors** whose **inner product** is the **sum** of the **pairwise products** of the elements
- **create** and **array/vector**

$$v^{k_1, k_2} \in \mathbb{R}^{N_1 \times N_2}$$

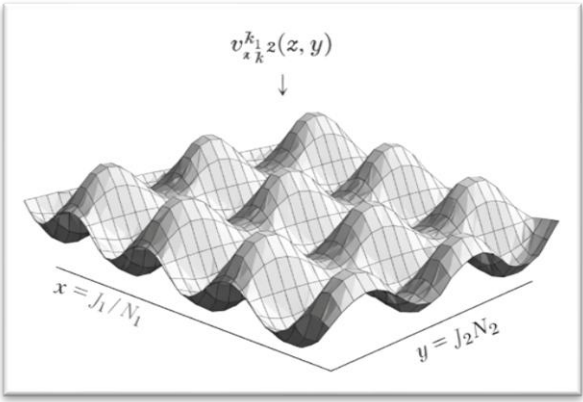
- with **two dimensions**

$$v_{j_1, j_2}^{k_1, k_2} = \exp \left(i \frac{2\pi j_1 k_1}{N_1} \right) \exp \left(i \frac{2\pi j_2 k_2}{N_2} \right)$$

Visualizing
Waves

- this **vector** is both **two-dimensional** as well as **complex**
- the **components** are **waves**

$$v_{j_1, j_2}^{k_1, k_2} = \sin \left(\frac{2\pi j_1 k_1}{N_1} \right) \sin \left(\frac{2\pi j_2 k_2}{N_2} \right)$$

	• normalized index coordinates $\left(\frac{j_1}{N_1}, \frac{j_2}{N_2}\right)$ • as a point in a unit square $x = \frac{j_1}{N_1}, \quad y = \frac{j_2}{N_2}$ • continuous domain representation $v^{k_1, k_2}(x, y) = \sin(2\pi k_1 x) \sin(2\pi k_2 y)$  ChatGPT 5.2.1
Orthogonal Vectors	• the vectors are orthogonal to each other $v_{j_1 j_2}^{k_1, k_2} = \exp\left(i \frac{2\pi j_1 k_1}{N_1}\right) \exp\left(i \frac{2\pi j_2 k_2}{N_2}\right)$ • orthogonality condition $v^{r_1, r_2} \cdot v^{s_1, s_2} = 0$ if $(r_1, r_2) \neq (s_1, s_2)$ • and the length of all the vectors are the same (vector norm) $ v^{k_1, k_2} ^2 = v^{k_1, k_2} \cdot v^{k_1, k_2} = N_1 N_2$ • the dot product is defined by listing all of the entries in a single list
Projection	
Solution	• the closest point p has the property that, $(x - p) \cdot v = 0$

	- for every vector $\mathbf{v}$ in $\mathbf{V}$, but that happens if, $(\mathbf{x} - \mathbf{p}) \cdot \mathbf{v}_i = 0$ - for every vector in the basis for $\mathbf{V}$, $\mathbf{v}_i^T \mathbf{x} = \mathbf{v}_i^T \mathbf{p}$ - since, $\mathbf{A} = [\mathbf{v}_1 \cdots \mathbf{v}_n]$ - that means, $\mathbf{A}^T \mathbf{x} = \mathbf{A}^T \mathbf{p}$
QR Factorization	- to help with this, we use a very important factorization — QR. - given a matrix $\mathbf{A}$ with linearly independent columns, we can factor $\mathbf{A} = \mathbf{Q}\mathbf{R}$ - example: $\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 0 & 1 \\ 2 & 0 \\ 2 & 2 \end{bmatrix} = \mathbf{Q}\mathbf{R}$ $\mathbf{Q} = \frac{1}{\sqrt{3}} \begin{bmatrix} 1 & 1 \\ 0 & 1 \\ 1 & -1 \\ 1 & 1 \end{bmatrix}, \mathbf{R} = \sqrt{3} \begin{bmatrix} 2 & 1 \\ 0 & 1 \end{bmatrix}$ $\mathbf{v}_1 = r_{11}\mathbf{q}_1$ $\mathbf{v}_2 = r_{12}\mathbf{q}_1 + r_{22}\mathbf{q}_2$
Julia	<pre> julia> using LinearAlgebra julia> A = [2 1; 0 1; 2 0; 2 2]; julia> qrf = qr(A); julia> Matrix(qrf.Q) 4×2 Matrix{Float64}: -0.57735 2.29738e-17 -0.57735 -0.57735 -0.57735 0.57735 -0.57735 -0.57735 julia> R = qrf.R 2×2 Matrix{Float64}: -3.4641 -1.73205 0.0 -1.73205 </pre>

	<pre>julia> Q = qrf.Q 4×2 Matrix{Float64}: 2.0 1.0 0.0 1.0 2.0 0.0 2.0 2.0 julia> Q'*Q 2×2 Matrix{Float64}: 1.0 -2.01213e-17 -2.01213e-17 1.0</pre>
Use QR for Projection	- combine $A^T z = A^T x$ $A = QR$ - get, $R^T Q^T z = R^T Q^T x$ $Q^T z = Q^T p$ - p is in the span of Q columns $p = Qc$ $Q^T z = Q^T Qc = c$ $p = Qc = QQ^T z$
Projection Onto a Subspace	
Challenge	- look at a subspace in n dimensional space, spanned by k linearly independent vectors (a basis) $V = \text{span}(v_1, v_2, \dots, v_k) \in \mathbb{R}^n$ - this is a k-dimensional subspace - for a point $x \in \mathbb{R}^n, x \notin V$, find the point in the subspace V that is closest to x - first observation is that every point in the subspace is of the form $p = Ac, \quad c \in \mathbb{R}^k$ $A = [v_1, \dots, v_n]$

Solution	- The closest point $\mathbf{p}$ has the property that $(\mathbf{x} - \mathbf{p}) \cdot \mathbf{v} = 0$ - for every vector $\mathbf{v}$ in $\mathbf{V}$ - but that happens if $(\mathbf{x} - \mathbf{p}) \cdot \mathbf{v}_i = 0$ - for every vector $\mathbf{v}$ in the basis for $\mathbf{V}$ $\mathbf{v}_i^T \mathbf{x} = \mathbf{v}_i^T \mathbf{p}$ since, $\mathbf{A} = [\mathbf{v}_1, \dots, \mathbf{v}_n]$ - that means: $\mathbf{A}^T \mathbf{x} = \mathbf{A}^T \mathbf{p}$
QR Factorization	- given a matrix $\mathbf{A}$, with linearly independent columns $\mathbf{A} = \mathbf{Q}\mathbf{R}$ - ex: $\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 0 & 1 \\ 2 & 0 \\ 2 & 2 \end{bmatrix} = \mathbf{Q}\mathbf{R}, \quad \mathbf{Q} = \frac{1}{\sqrt{3}} \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & -1 \\ 1 & 1 \end{bmatrix}, \quad \mathbf{R} = \sqrt{3} \begin{bmatrix} 2 & 1 \\ 0 & 1 \end{bmatrix}$
Julia	<pre> julia> using LinearAlgebra julia> A = [2 1; 0 1; 2 0; 2 2]; julia> qrf = qr(A) julia> Matrix(qrf.Q) 4x2 Matrix{Float64}: -0.57735 2.29938e-17 0.0 -0.57735 -0.57735 0.57735 -0.57735 -0.57735 julia> R = qrf.R 2x2 Matrix{Float64}: -3.4641 -1.73205 0.0 -1.73205 julia> Q*R 4x2 Matrix{Float64}: </pre>

```
2.0  1.0
0.0  1.0
2.0  4.85078e-18
2.0  2.0
```

```
julia> Q'*Q
2x2 Matrix{Float64}:
 1.0      -2.01213e-17
-2.01213e-17  1.0
```

Use QR for Projection

we want a **vector** p that
'behaves' like x when
'viewed through' the
columns of A

$$A^T x = A^T p$$

$$A = QR$$

- in **least squares** this translates to:
' p is the **closest vector** to x that lives
inside the column space of A '

choose a Q
(**orthogonal**) and R
(**upper triangular**)
such that they are a
decomposition of
invertible **matrix** A

- now, **replace A** with its **QR Factorization**

$$R^T Q^T x = R^T A^T$$

$$Q^T x = Q^T p$$

- since p lies in the **span** of the **columns** of Q ,

$$p = Qc$$

- for some **coefficient vector** c giving us,

$$Q^T x = Q^T Qc$$

- because Q is **orthogonal**

$$Q^T Q = I$$

Identity Matrix

- **so,**

$$c = Q^T x.$$

- **substituting** back, we get,

$$p = QQ^T x$$

translates to: ' p is the
projection of x onto
the column space of A

- what does it all mean?
 - starting with a **vector** x that **may or may not** lie in the
column space of A , the **least-squares** solution finds the
vector p inside that **column space** that is **closest** to x .

Julia

```
julia> using LinearAlgebra

julia> A = [2 1;0 1;2 0;2 2];

julia> qrf = qr(A);

julia> x = [1,2,3,4];

julia> Q = Matrix(qrf.Q);

julia> p = Q*Q'*x
4-element Vector{Float64}:
 2.6666666666666665
 0.9999999999999999
 1.6666666666666667
 3.6666666666666665

julia> A'*(p-x)
2-element Vector{Float64}:
 -4.440892098500626e-16
 -4.440892098500626e-16

julia> norm(x)
5.477225575051661

julia> norm(p)
4.93288262316247

julia> norm(p-x)
2.3804761428476167

julia> sqrt(4.93^2+2.38^2)
5.4742234395557
```

Least Squares

Challenge

- you **have** data **points**

$$(x_1, y_1), (x_2, y_2), \dots$$

- you **want** to **approximate** as a single **line**

$$f(x) = a + bx$$

basic **line equation**:
 a = **y-intercept**
 b = **slope**

- ideally**, each data point **would satisfy**

$$y = a + bx_i$$

- the **expectation** of this as a **solution** is **unrealistic**, so **instead**

$$y_i \approx a + bx_i$$

meaning: the line
should be pretty close

Rewriting the System of Equations as a Matrix

- instead** of **writing** an **equation** for each point, we **stack** them

all of the y_i values
go into **one column**

$$\begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \\ y_5 \end{bmatrix} = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ 1 & x_3 \\ 1 & x_4 \\ 1 & x_5 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix}$$

linear expressions

- so, the **system becomes**

$$y \approx A \begin{bmatrix} a \\ b \end{bmatrix}$$

compactly **represents**
all equations in the
system

The Least Squares Solution

- finding a **single** choice of a and b to **satisfy** our **equation** is **unlikely**
- instead**, we want to find a and b that make the **total error** as **small** as **possible**

- for each point **compute** the **vertical distance**

$$error_i = y_i - (a + bx_i)$$

- then, we **minimize** the **error**

$$\sum (error_i)^2$$

why **square** the **errors**?

- negative and positive **errors** **do not cancel**
- large** errors **penalize more**

Residual

- the **residual** is simply *the error at each data point*, expressed as

$$r = y - Ac$$

y = measured values

Ac = predicted values

r = the **difference** between measured and predicted

- so, **for each point**

$$\text{residual} = \text{actual value} - \text{predicted value}$$

- expressed as the **matrix equation**

$$r = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \\ y_5 \end{bmatrix} - \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ 1 & x_3 \\ 1 & x_4 \\ 1 & x_5 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix}$$

- which **computes** for every **data point**

$$y_i - (a + bx_i)$$

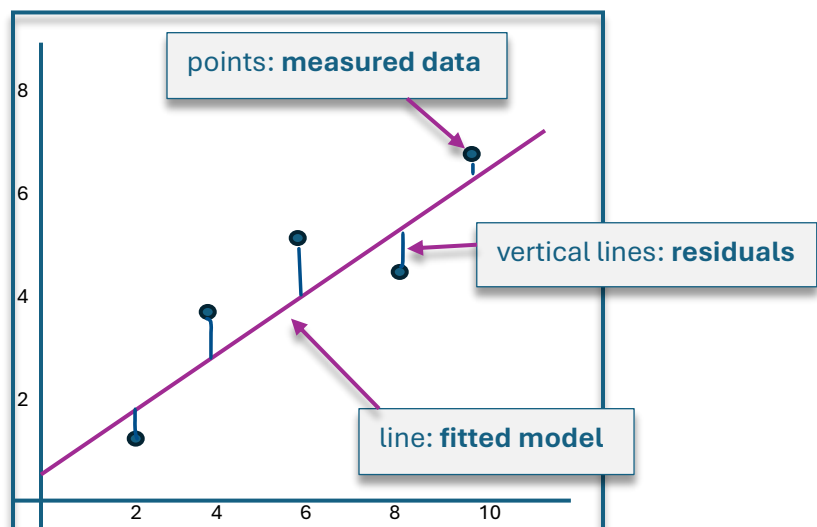
- conceptually**, this means the **data** (y) **equals** the **fitted line** (Ac) **plus error** (r), or

$$y = Ac + r$$

y = the **data**

Ac = the **model**

r = the '**leftover**' part



Normal Equations

- we are trying to **solve**

$$Ac \approx y$$

- but, **usually** there are **more equations** than **knowns** meaning **no exact solutions** exist

- so, **instead** we ask: *which vector makes Ac as close as possible to y ?*

- to find the “close as possible” solution we **measure** the **error** using the **residual**

$$r = y - Ac, \text{ minimizing } ||r||$$

primary goal

- the **key geometric** idea, the best **approximation** $p = Ac$ happens when

normal equations

$$A^T(y - p) = 0$$

this says:
“the **error** must be **perpendicular** to the **column space**”

- turning that into **equations**, if $p = Ac$

$$A^T y = A^T A c$$

projection **condition**

- solving for c , if $A^T A$ is **invertible**

$$c = (A^T A)^{-1} A^T y$$

gives least sq. **solution**

- what** this **accomplishes**

- makes the **residual** **perpendicular** to the **columns** of A
- **minimizes** the **size** of the **residual** vector
- gives the **best approximation** inside the **column space**

Julia

```
julia> A = [1 1;2 1;3 1;4 1;6 1];
```

```
julia> y = [2 4 5 4 7 0]';
```

```
julia> c = (A'*A)\(A'*y)
```

```
2x1 Matrix{Float64}:
```

```
0.8513513513513513
```

```
1.0
```

```
julia> c = A\y
```

```
2x1 Matrix{Float64}:
```

```
1.6756756756756768
```

	<pre>0.8513513513513514 julia> r = y-A*c 5x1 Matrix{Float64}: -0.5270270270270281 0.6216216216216202 0.7702702702702684 -1.0810810810810825 -0.8108108108108102</pre>
Generalization	- so far, we have discussed straight lines, but least squares can be applied to more complicated models - this is clearly the equation of a parabola - we still solve - we still compute $f(x) = a + bx + cx^2$ $A^T A c = A^T y$ $c = (A^T A)^{-1}$ a = intercept b = linear slope c = curvature this holds true for any degree polynomial